

Theory of Programming Languages

3. Simply Typed Lambda Calculus

Kihong Heo



Type

- Role of types in PL: framework for understanding and designing PLs
 - Syntax: constructors for each type
 - Semantics: coherence between static and dynamic semantics
- Harmony between two aspects of programs
 - Dynamic semantics (동적, 실행의미): runtime behavior (e.g., operational semantics)
 - Static semantics (정적, 실행전 의미): estimation of runtime behavior (e.g., type system)

Simply Typed Lambda Calculus

단순 타입 람다계산법

- Introduced by Alonzo Church (1940)
- Restricted and conservative: ill-typed terms excluded
- Syntax: type annotation for all lambda abstraction
- Base type unit or function type

$$\begin{array}{lcl} \text{Term } E & \rightarrow & x \mid \lambda x:\tau.E \mid E E \mid \langle \rangle \\ \text{Type } \tau & \rightarrow & \text{unit} \mid \tau \rightarrow \tau \end{array}$$

Dynamic Semantics 동적 의미구조

- Term, type, context, value

$$\begin{array}{llll} \text{Term} & E & \rightarrow & x \mid \lambda x : \tau. E \mid E E \mid \langle \rangle \\ \text{Type} & \tau & \rightarrow & \text{unit} \mid \tau \rightarrow \tau \\ \text{Ctx} & K & \rightarrow & [] \mid K E \mid v K \\ \text{Value} & v & \rightarrow & \langle \rangle \mid \lambda x : \tau. E \end{array}$$

- Operational semantics

$$\frac{}{(\lambda x : \tau. E) v \rightarrow E\{v/x\}}$$

$$\frac{E \rightarrow E'}{K[E] \rightarrow K[E']}$$

Static Semantics 정적 의미구조

- Proof of judgement $\Gamma \vdash E : \tau$ means “ E has type τ in environment Γ ”
- Type environment $\Gamma \in \text{Var} \rightarrow \text{Type}$
- Typing rules

$$\frac{}{\Gamma \vdash \langle \rangle : \text{unit}} \quad \frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau} \quad \frac{\Gamma + x : \tau_1 \vdash E : \tau_2}{\Gamma \vdash \lambda x : \tau_1 . E : \tau_1 \rightarrow \tau_2} \quad \frac{\Gamma \vdash E_1 : \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash E_2 : \tau_1}{\Gamma \vdash E_1 E_2 : \tau_2}$$

Dynamic $\approx$ Static

- Consistency between dynamic and static semantics?
 - Soundness (안전성): “well-typed” (static) $\implies$ “well-behaved” (dynamic)
 - Completeness (완전성): “well-behaved” (dynamic) $\implies$ “well-typed” (static)
- Impossible to achieve both (i.e., undecidable, 계산불가능성)
- Typically, sound type systems are used

Soundness

Theorem (Soundness). For all terms E, E' , if $\vdash E : \tau$ and $E \rightarrow^* E'$, then $E' \in \text{Value}$ or $\exists E'' . E' \rightarrow E''$

Lemma (Preservation). For all terms E, E' , if $\vdash E : \tau$ and $E \rightarrow E'$, then $\vdash E' : \tau$

Lemma (Progress). For all terms E , if $\vdash E : \tau$ then $E \in \text{Value}$ or $\exists E' . E \rightarrow E'$

Enriching the Language

- Extend the language with values and their types
- Add new types = provide ways for construction and use for each type
 - Introduction/elimination (production/consumption)
- Example: function type
 - Introduction: $\lambda x : \tau . E$ (abstraction)
 - Elimination: $E E$ (application)

Let Binding

- Terms and evaluation contexts

$$\begin{array}{lcl} E & \rightarrow & \dots \mid \text{let } x = E \text{ in } E \\ K & \rightarrow & \dots \mid \text{let } x = K \text{ in } E \end{array}$$

- Dynamic semantics

$$\frac{E \rightarrow E'}{K[E] \rightarrow K[E']}$$

$$\frac{}{\text{let } x = v \text{ in } E \rightarrow E\{v/x\}}$$

- Static semantics

$$\frac{\Gamma \vdash E_1 : \tau_1 \quad \Gamma + x:\tau_1 \vdash E_2 : \tau_2}{\Gamma \vdash \text{let } x = E_1 \text{ in } E_2 : \tau_2}$$

Product Type (1)

- Terms, evaluation contexts, values, and types

$$E \rightarrow \dots \mid \langle E, E \rangle \mid E.1 \mid E.2$$

$$K \rightarrow \dots \mid \langle K, E \rangle \mid \langle v, K \rangle \mid K.1 \mid K.2$$

$$v \rightarrow \dots \mid \langle v, v \rangle$$

$$\tau \rightarrow \dots \mid \tau \times \tau$$

- Dynamic semantics

$$\frac{E \rightarrow E'}{K[E] \rightarrow K[E']}$$

$$\frac{}{\langle v_1, v_2 \rangle.1 \rightarrow v_1} \quad \frac{}{\langle v_1, v_2 \rangle.2 \rightarrow v_2}$$

Product Type (2)

- Static semantics

$$\frac{\Gamma \vdash E_1 : \tau_1 \quad \Gamma \vdash E_2 : \tau_2}{\Gamma \vdash \langle E_1, E_2 \rangle : \tau_1 \times \tau_2}$$

$$\frac{\Gamma \vdash E : \tau_1 \times \tau_2}{\Gamma \vdash E.1 : \tau_1} \quad \frac{\Gamma \vdash E : \tau_1 \times \tau_2}{\Gamma \vdash E.2 : \tau_2}$$

Tuple Type (1)

- Generalization of binary products (i.e., n-ary product)
- Terms, evaluation contexts, values, and types

$$E \rightarrow \dots \mid \langle E_i^{i \in 1 \dots n} \rangle \mid E.i$$

$$K \rightarrow \dots \mid \langle v_i^{i \in 1 \dots j-1}, K, E_k^{k \in j+1 \dots n} \rangle \mid K.i$$

$$v \rightarrow \dots \mid \langle v_i^{i \in 1 \dots n} \rangle$$

$$\tau \rightarrow \dots \mid \langle \tau_i^{i \in 1 \dots n} \rangle$$

- Dynamic semantics

$$\frac{E \rightarrow E'}{K[E] \rightarrow K[E']}$$

$$\frac{}{\langle v_i^{i \in 1 \dots n} \rangle.j \rightarrow v_j}$$

Tuple Type (2)

- Static semantics

$$\frac{\text{for each } i \quad \Gamma \vdash E_i : \tau_i}{\Gamma \vdash \langle E_i \mid i \in 1 \dots n \rangle : \langle \tau_i \mid i \in 1 \dots n \rangle}$$

$$\frac{\Gamma \vdash E : \langle \tau_i \mid i \in 1 \dots n \rangle}{\Gamma \vdash E.j : \tau_j}$$

Record Type (1)

- Generalization of tuples (i.e., tuples with labels)
- Terms, evaluation contexts, values, and types

$$E \rightarrow \dots \mid \{ \mathbf{l}_i = E_i \mid i \in 1 \dots n \} \mid E. \mathbf{l}$$

$$K \rightarrow \dots \mid \{ \mathbf{l}_1 = v_1 \mid i \in 1 \dots j-1, \mathbf{l}_j = K, \mathbf{l}_k = E_k \mid k \in j+1 \dots n \} \mid K. \mathbf{l}$$

$$v \rightarrow \dots \mid \{ \mathbf{l}_i = v_i \mid i \in 1 \dots n \}$$

$$\tau \rightarrow \dots \mid \{ \mathbf{l}_i : \tau_i \mid i \in 1 \dots n \}$$

- Dynamic semantics

$$\frac{E \rightarrow E'}{K[E] \rightarrow K[E']}$$

$$\frac{}{\{ \mathbf{l}_i = v_i \mid i \in 1 \dots n \}. \mathbf{l}_j \rightarrow v_j}$$

Record Type (2)

- Static semantics

$$\frac{\text{for each } i \quad \Gamma \vdash E_i : \tau_i}{\Gamma \vdash \{1_i = E_i \mid i \in 1 \dots n\} : \{1_i : \tau_i \mid i \in 1 \dots n\}}$$

$$\frac{\Gamma \vdash E : \{1_i : \tau_i \mid i \in 1 \dots n\}}{\Gamma \vdash E.1_j : \tau_j}$$

Sum Type (1)

- Terms, evaluation contexts, values, and types

$$\begin{aligned} E &\rightarrow \dots \mid \text{inl } E \mid \text{inr } E \\ &\quad \mid \text{case } E \text{ of } \text{inl } x \Rightarrow E \mid \text{inr } x \Rightarrow E \\ K &\rightarrow \dots \mid \text{inl } K \mid \text{inr } K \\ &\quad \mid \text{case } K \text{ of } \text{inl } x \Rightarrow E \mid \text{inr } x \Rightarrow E \\ v &\rightarrow \dots \mid \text{inl } v \mid \text{inr } v \\ \tau &\rightarrow \dots \mid \tau + \tau \end{aligned}$$

- Dynamic semantics

$$\frac{E \rightarrow E'}{K[E] \rightarrow K[E']}$$

$$\frac{}{\text{case (inl } v) \text{ of } \text{inl } x \Rightarrow E_1 \mid \text{inr } x \Rightarrow E_2 \rightarrow E_1\{v/x\}}$$

$$\frac{}{\text{case (inr } v) \text{ of } \text{inl } x \Rightarrow E_1 \mid \text{inr } x \Rightarrow E_2 \rightarrow E_2\{v/x\}}$$

Sum Type (2)

- Static semantics

$$\frac{\Gamma \vdash E : \tau_1}{\Gamma \vdash \text{inl } E : \tau_1 + \tau_2} \quad \frac{\Gamma \vdash E : \tau_2}{\Gamma \vdash \text{inr } E : \tau_1 + \tau_2}$$
$$\frac{\Gamma \vdash E : \tau_1 + \tau_2 \quad \Gamma + x : \tau_1 \vdash E_1 : \tau \quad \Gamma + x : \tau_2 \vdash E_2 : \tau}{\Gamma \vdash \text{case } E \text{ of inl } x \Rightarrow E_1 \mid \text{inr } x \Rightarrow E_2 : \tau}$$

- Problem?
- Example: What is the type of `inl 5` ?

Sum Type (3)

- Explicit type annotation to sum type

$$\begin{aligned} E &\rightarrow \dots \mid \text{inl } E \text{ as } \tau \mid \text{inr } E \text{ as } \tau \\ &\quad \mid \text{case } E \text{ of inl } x \Rightarrow E \mid \text{inr } x \Rightarrow E \\ K &\rightarrow \dots \mid \text{inl } K \text{ as } \tau \mid \text{inr } K \text{ as } \tau \\ &\quad \mid \text{case } K \text{ of inl } x \Rightarrow E \mid \text{inr } x \Rightarrow E \\ v &\rightarrow \dots \mid \text{inl } v \text{ as } \tau \mid \text{inr } v \text{ as } \tau \\ \tau &\rightarrow \dots \mid \tau + \tau \end{aligned}$$

- Static semantics

$$\frac{\Gamma \vdash E : \tau_1}{\Gamma \vdash \text{inl } E \text{ as } \tau_1 + \tau_2 : \tau_1 + \tau_2} \quad \frac{\Gamma \vdash E : \tau_2}{\Gamma \vdash \text{inr } E \text{ as } \tau_1 + \tau_2 : \tau_1 + \tau_2}$$
$$\frac{\Gamma \vdash E : \tau_1 + \tau_2 \quad \Gamma + x : \tau_1 \vdash E_1 : \tau \quad \Gamma + x : \tau_2 \vdash E_2 : \tau}{\Gamma \vdash \text{case } E \text{ of inl } x \Rightarrow E_1 \mid \text{inr } x \Rightarrow E_2 : \tau}$$

Variant Type (1)

- Generalization of binary sums
- Terms, evaluation contexts, values, and types

$$\begin{aligned} E &\rightarrow \dots \mid \mathbf{1} \ E \text{ as } \tau \mid \text{case } E \text{ of } \mathbf{1}_i \ x_i \Rightarrow E_i^{i \in 1 \dots n} \\ K &\rightarrow \dots \mid \mathbf{1} \ K \text{ as } \tau \mid \text{case } K \text{ of } \mathbf{1}_i \ x_i \Rightarrow E_i^{i \in 1 \dots n} \\ v &\rightarrow \dots \mid \mathbf{1} \ v \text{ as } \tau \\ \tau &\rightarrow \dots \mid \langle \mathbf{1}_i : \tau_i^{i \in 1 \dots n} \rangle \end{aligned}$$

- Dynamic semantics

$$\frac{E \rightarrow E'}{K[E] \rightarrow K[E']}$$

$$\frac{}{\text{case } \mathbf{1}_j \ v_j \text{ of } \mathbf{1}_i \ x_i \Rightarrow E_i^{i \in 1 \dots n} \rightarrow E_j\{v_j/x_j\}}$$

Variant Type (2)

- Static semantics

$$\frac{\Gamma \vdash E_j : \tau_j}{\Gamma \vdash \mathbf{1}_j E_j \text{ as } \langle \mathbf{1}_i : \tau_i \mid i \in 1 \dots n \rangle : \langle \mathbf{1}_i : \tau_i \mid i \in 1 \dots n \rangle}$$
$$\frac{\Gamma \vdash E : \langle \mathbf{1}_i : \tau_i \mid i \in 1 \dots n \rangle \quad \text{for each } i \quad \Gamma + x_i : \tau_i \vdash E_i : \tau}{\Gamma \vdash \text{case } E \text{ of } \mathbf{1}_i x_i \Rightarrow E_i \mid i \in 1 \dots n : \tau}$$

Recursion

- Type of $\Omega = (\lambda x.x\ x)(\lambda x.x\ x)$
- Type of $\text{fix} = \lambda f.(\lambda x.f\ (\lambda y.x\ x\ y))\ (\lambda x.f\ (\lambda y.x\ x\ y))$
- Type of $(x\ x)$

Inhabitation

- If there exists a term E such that $\emptyset \vdash E : \tau$ then E is an inhabitant of τ
 - $\text{unit} : \{\langle \rangle\}$
 - $\text{bool} : \{\text{true}, \text{false}\}$
 - $\text{nat} : \{0, 1, 2, \dots\}$
 - $\text{nat} \rightarrow \text{nat} : \{\text{succ}, \dots\}$
- Uninhabited type?
 - $\tau : \emptyset ?$

Void Type (1)

- Uninhabited type (i.e., no values of void type exist)
 - Empty type
 - Dual of unit type: unit has only one value, whereas void has none
- No values and dynamic semantics

$$\begin{array}{lcl} E & \rightarrow & \dots \mid \text{abort}_\tau E \\ \tau & \rightarrow & \dots \mid \text{void} \end{array} \qquad \text{abort}_\tau E = \text{case } E \text{ of } \mid$$

- Static semantics (only elimination)

$$\frac{\Gamma \vdash E : \text{void}}{\Gamma \vdash \text{abort}_\tau E : \tau}$$

Void Type (2)

- Example: $\lambda x : \text{void} + \text{unit}. \text{case } x \text{ of } \text{inl } y \Rightarrow \text{abort}(y) \mid \text{inr } y \Rightarrow y$
- “Nothing” in Scala, “never” in Rust
- Simulation in OCaml

```
type void = |
type ('a, 'e) t = 0k of 'a | Error of 'e
type ('a, 'e) operation = int -> int -> ('a, 'e) t

let add : (int, void) operation = fun x y -> 0k (x + y)
let sub : (int, void) operation = fun x y -> 0k (x - y)
let mul : (int, void) operation = fun x y -> 0k (x * y)
let div : (int, string) operation =
  fun x y -> if y = 0 then Error "Division by zero" else 0k (x / y)

let log_result name op x y =
  match op x y with
  | 0k v -> Printf.printf "%s(%d, %d) = %d\n" name x y v; 0k v
  | Error e -> Printf.printf "%s(%d, %d) failed\n" name x y; Error e

let _ = log_result "add" add 10 5
let _ = log_result "div" div 10 0
```

```
let run_infallible (op : ('a, void) operation) x y =
  match op x y with 0k v -> v | Error impossible -> .

let run_fallible op x y =
  match op x y with
  | 0k v -> v
  | Error e -> prerr_endline e; exit 1

let _ = run_infallible add 10 5
let _ = run_fallible div 10 0
```

Normalization

- Every well-typed term is normalizable in STLC
 - The evaluation of a well-typed program eventually halts
- Not true in general
- Proof! (Excecise)

Summary

- Simple types: unit, void, function, product, sum, etc
- Conservative but restricted: reject many “meaningful” programs
- Normalization: the evaluation of a well-typed program eventually halts